Chapter 4

Deep fully-connected networks, Backpropagation

Reading

1. Bishop 5.1, 5.3
2. Bishop DL 6.1-6.3.3, Chapter 8
3. Goodfellow 6.3-6.5
4. Notes at http://cs231n.github.io/optimization-2/

4.1 Deep fully-connected networks

A deep neural network takes the idea of a two-layer network to the next step. Instead of having one matrix S in the classifier

$$f(x; v, S) = \text{sign}\left(v^\top \sigma\left(S^\top x\right)\right)$$

a deep network has many matrices $S_1, \ldots, S_L$

$$f(x; v, S_1, \ldots, S_L) = \text{sign}\left(v^\top \sigma\left(S_L^\top \ldots \sigma\left(S_2^\top\ \sigma(S_1^\top x)\right) \ldots\right)\right). \quad (4.1)$$

We will call each operation of the form $\sigma\left(S_k^\top \ldots\right)$, a *layer*. Consider the second layer: it takes the features generated by the first layer, namely $\sigma(S_1^\top x)$, multiplies these features using its feature matrix $S_2^\top$ and applies a nonlinear function $\sigma(\cdot)$ to this result element-wise before passing it on to the third layer.

A deep network creates new features by composing older features.

This composition is very powerful. Not only do we not have to pick a particular feature vector, we can create very complex features by sequentially combining simpler ones. For example Fig. 4.1 shows the features (more precisely, the kernel) learnt by a deep neural network. The first layer of features are called Gabor-like, they are similar to ones you constructed in HW 1. These features are *combined* linearly along with a nonlinear operation to give richer features (spirals, right angles) in the middle panel. The third layer combines the lower features to get even more complex features, these look like patterns (notice a soccer ball in the bottom left), a box on the bottom right etc.

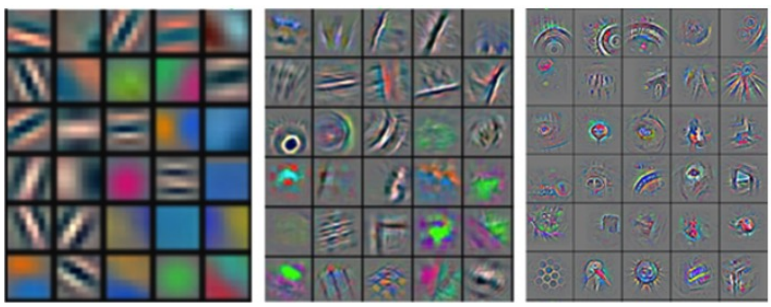


Figure 4.1

The optimization problem for fitting a deep network is written as

$$v^*, S_1^*, \ldots, S_L^* = \underset{v, S_1, \ldots, S_L}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \ell_{\text{hinge}}(y^i, \hat{y}^i) \tag{4.2}$$

where the output prediction is

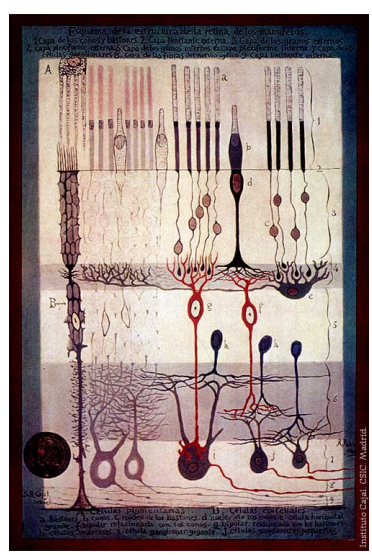
$$\hat{y} = v^\top \sigma\left(S_L^\top \ldots \sigma\left(S_2^\top \sigma(S_1^\top x)\right) \ldots\right).$$

Notice that if fitting a two-layer network was difficult, then fitting a multi-layer neural network like Eq. (4.1) is even harder. There are *lots* of parameters and consequently we need a lot more data to fit such a model. The optimization problem in Eq. (4.2) is also naturally much harder than its two-layer version. The benefit for going through this difficulty is many fold and quite astounding.

1. Not having to pick features is very powerful. Notice that we do not need to worry about what kind of data x is at the input. So long as we can write it into a vector, the classifier as written in Eq. (4.1) works. In other words, the same type of classifier works for image-based data, data from natural language processing, speech processing, and many other types. This is the primary reason why a large number of scientific fields are adopting deep networks.

The mammalian retina Circuits in the retina are hard-wired at birth because being able to see is so important to survival; there is no learning in the retina itself although there is a clear hierarchy of neurons that successively process information. Later parts of the visual cortex get learned during your lifetime.

The retina transcribes photons that are incident upon the eye using rod cells (function better in low light) and cone cells (function better in bright conditions). This is further processed by "bipolar" cells into action potentials, or "spikes". Amacrine cells make lateral, inhibitory connections to remove redundancy in the stimuli. Ganglion cells create ~20 visual features (edges/spots, local motions at 90°/120° angles, colors, etc.). Altogether, ~80 types of neurons transmit ~10 Mbps of information to the brain. These neurons are surprisingly similar to each other, e.g., all cell types fire at 4-8 Hz and different ganglion cells learn highly redundant features. Read Balasubramanian (2015) for an exciting description of why neural circuits are wired the way they are.



A picture of the neurons in the retina drawn by Santiago Ramón y Cajal using a microscope in the 1900s.

2. Before the resurgence of deep learning, each of these fields essentially had their own favorite kernels they preferred, these kernels were designed across decades of insights from that specific field (wavelets in signal processing, keypoint detectors and descriptors in computer vision, n-grams in NLP etc.). It was very difficult for a researcher to use ideas from a different field. With deep learning, this has become much easier. There is still a significant amount of domain insight that you need to make deep networks work well but the bar for entering a new field is much lower.

3. Deep neural networks are universal approximators. In simple words, it means that provided the deep network has enough number of layers and enough number of features in each layer, it can fit any dataset. This is a theorem in approximation theory.

4.1.1 Some deep learning jargon

We have defined the essential parts of a deep network. Let us briefly take a look at some typical jargon you will encounter as you read more.

Activation function. The nonlinear function $\sigma(\cdot)$ in Eq. (4.1) is called the activation function (motivated from the threshold-based activation of McCulloch-Pitts neuron). It is also called a nonlinearity because it is the only nonlinear operation in the classifier. There are many activation functions that have been used over the years.

1. Threshold
$$\text{threshold}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{else.} \end{cases}$$

2. Sigmoid/Logistic
$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}.$$

3. Hyperbolic tangent
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

4. Rectified Linear Units (ReLU)
$$\begin{aligned} \text{relu}(x) &= |x|_+ \\ &= \max(0, x). \end{aligned}$$

5. Leaky ReLUs
$$\sigma_c(x) = \begin{cases} x & \text{if } x > 0 \\ c\,x & \text{else.} \end{cases}$$

6. Swish
$$\sigma(x) = x\ \text{sigmoid}(x).$$

Different activation functions work differently. ReLU nonlinearities are the most popular and we will see the reasons why they work better than older ones such as sigmoid/tanh nonlinearities in the backpropagation section.

Logits for multi-class classification. The output

$$\hat{y} = v^\top \sigma\left(S_L^\top \ldots \sigma\left(S_2^\top\ \sigma(S_1^\top x)\right)\ldots\right)$$

are called the logits corresponding to the different classes. This name comes from logistic regression where logits are the log-probabilities of belonging to one of the two classes. A deep network affords an easy way to solve a multi-class classification problem, we simply set

$$v \in \mathbb{R}^{p\times C}$$

where C is the total number of classes in the data. Just like logistic regression predicts the logits of the two classes, we would like to *interpret* the vector $\hat{y}$ as the log-probabilities of an input belonging to one of the classes.

❓ How would you use a binary classifier to classify 10 classes?

❓ What would the shape of v be if you were performing regression using a deep network?

Mid-level features. The features at any layer can be studied once you create a deep network. You pass an input image x and compute

$$h^l = S_l^\top \ldots \sigma\left(S_2^\top\ \sigma(S_1^\top x)\right)\ldots \tag{4.3}$$

to get the *pre-activation* output of the lth layer. The *post-activation* output is given by applying the nonlinearity

$$\sigma(h^l).$$

Sometimes people will call the $\sigma(h^L)$ as the *feature* created by a deep network; the rationale here is that just like a kernel-based classifier uses features $\phi(x)$ and fits a linear classifier to these features we may think of the feature of a deep network to be $\sigma(h^L)$. These features are often very useful, e.g., you can use the lower layers of a deep network trained on a different dataset, say classifying cats vs. dogs, as the feature generator but retrain the classifier weights v on your specific problem, say classifying apples vs. oranges. Such pre-training is typically used to exploit the fact that someone else has trained a large deep network on a large dataset, and thereby learnt a rich feature generator. Training the large model yourself on a large dataset like ImageNet would be quite difficult.

Hidden layers/neurons. The intermediate layers that create the features $h^1, \ldots, h^L$ are called the hidden layers. A feature is the same as a neuron; think of the McCulloch-Pitts picture, just like a neuron takes input from all the other neurons connected to it via some weights , a feature is computed using a weighted combination of the features at the lower layer. We will say that a neural network is *wide* if it has lots of features/neurons on each

hidden layer. We will say that it is *thin* if it has few features/neurons on each hidden layer.

4.1.2 Weights

It is customary to not differentiate between the parameters of different layers of a deep network and simply say *weights* when we want to refer to all parameters. The set

$$w := \{v, S_1, S_2, \ldots, S_L\}$$

is the set of *weights*. This set is typically stored in PyTorch as a set of matrices, one for each layer.

Important. Every time we want to write down mathematical equations, we will imagine w to be a large vector. This is less cumbersome notation. We denote by p the dimensionality of w and imagine that

$$w \in \mathbb{R}^p.$$

The dimensionality p keeps things consistent with linear classifiers where the features were $\phi(x) \in \mathbb{R}^p$. When you use PyTorch to implement an algorithm that requires you to iterate over the weights, say you were implementing SGD from scratch, you will iterate over elements of the set of weights. Using this new notation, we will write down a deep network as simply

$$f(x, w) \tag{4.4}$$

and fitting the deep network to a dataset involves the optimization problem

$$w^* = \underset{w}{\text{argmin}} \ \frac{1}{n} \sum_{i=1}^{n} \ell(y^i, \hat{y}^i; w). \tag{4.5}$$

We will often denote the loss of the ith sample as simply

$$\ell^i(w) := \ell(y^i, \hat{y}^i; w).$$

4.2 The backpropagation algorithm

We would like to using SGD to fit a deep network on a given dataset. As we saw in Chapter 2, if the loss function is denoted by $\ell^{\omega_t}(w)$ where ω_t was the index of the datum sampled at iteration t, we would like to update the weights using

$$w^{(t+1)} = w^{(t)} - \eta \frac{\mathrm{d}\ell^{\omega_t}(w)}{\mathrm{d}w} \Big|_{w=w^{(t)}}.$$

We have used a scalar $\eta > 0$ as the step-size or the learning rate. It governs the distance traveled along the negative gradient at each iteration. Let us ignore the index of the datum ω_t in this section, imagine $\omega_t = 1$. Implementing SGD therefore boils down to computing the gradient

$$\frac{\mathrm{d}\ell(w)}{\mathrm{d}w}.$$

> Backpropagation is an algorithm for computing the gradient of the loss function with respect to weights of a deep network.

4.2.1 One hidden layer with one neuron

Consider the linear regression problem with one layer and one datum, $w, x \in \mathbb{R}^d$ and $v, y \in \mathbb{R}$:

$$\ell(w, v) = \frac{1}{2}(y - v\sigma(w^\top x))^2$$

where $\sigma(\cdot)$ is some activation function and our weights are $\{v, w\}$. Let us understand the computational graph of how the loss is computed:

$$w, x \quad \underbrace{\longmapsto}_{\text{layer 1}} \quad z \quad \underbrace{\overset{\sigma}{\longmapsto}}_{\text{layer 2}} \quad h \quad \underbrace{\overset{v}{\longmapsto}}_{\text{layer 3}} \quad vh \quad \underbrace{\overset{y}{\longmapsto}}_{\text{layer 4}} \quad \ell. \tag{4.6}$$

where $h = \sigma(z)$ and $z = w^\top x$. Each node in this graph is either the input/output or an intermediate result of the computation. The gradient of the loss with respect to the weights using the chain rule is

$$\frac{\partial \ell}{\partial v} = (y - v\sigma(w^\top x))\left(-\sigma(w^\top x)\right) \tag{4.7}$$

and

$$\frac{\partial \ell}{\partial w} = (y - v\sigma(w^\top x))\left(-v\sigma'(w^\top x)\right)(x)\,. \tag{4.8}$$

> 1. **Caching computations for computing the chain rule.** The

first idea behind backpropagation is to realize that quantities like $(y - v\sigma(w^\top x))$ or $z = w^\top x$ are computed multiple times in the chain rule in Eqs. (4.7) and (4.8). If we can cache these quantities we can compute the chain rule-based gradient for the different parameters quickly.

2. **Cache is the output of each layer.** The second idea behind backpropagation is to realize that quantities like $(y - vh)$, $h = \sigma(z)$ and $z = w^\top x$ are outputs of the third, second and first layers respectively. In other words, the quantities we need to cache in the chain rule computation are simply the outputs of the individual layers.

3. **Derivatives of the loss with respect to the input of a layer only depends on what happens in that layer and the derivative of the loss with respect to the output of that layer.** The third observation is to see that the quantity $\sigma'(z)$ in Eq. (4.8) is the derivative of the output of the activation function, namely $h = \sigma(z)$ with respect to z, its input argument

$$\sigma'(z) = \frac{\mathrm{d}h}{\mathrm{d}z}.$$

This derivative is combined with the forward computation $(y - vh)$ to get the gradient with respect to the weights w.

Backpropagation is simply a book-keeping exercise that caches the forward computation of the graph in Eq. (4.6) and uses these cached values to compute the derivative of the loss ℓ with respect to the parameters of each layer sequentially.

We will use a clever notation to denote the backprop gradient which will make this process very easy. Denote by

$$\overline{v} = \frac{\mathrm{d}\ell}{\mathrm{d}v} \tag{4.9}$$

the derivative of the loss ℓ with respect to a parameter v. For our simple two layer (one neuron) neural network, we are interested in computing the quantities

$$\overline{w}, \quad \overline{v}.$$

Let us also denote the output of the second linear layer (layer 3) as

$$e = vh.$$

Now observe the following "forward computation"

$$z = w^\top x \tag{4.10}$$
$$h = \sigma(z) \tag{4.11}$$
$$e = vh \tag{4.12}$$
$$\ell = \frac{1}{2}\left(y - e\right)^2. \tag{4.13}$$

Let us imagine that we have cached all the quantities on the left hand side of the equalities above. We use these quantities to perform the "backward" computation as follows

$$\begin{aligned}
\frac{\mathrm{d}\ell}{\mathrm{d}\ell} &= \overline{\ell} = 1. \\
\mathbb{R} \ni \overline{e} &= \overline{\ell}\,\frac{\mathrm{d}\ell}{\mathrm{d}e} \\
&= -1\,(y-e) = \overline{\ell}\,(-(y-e))\,. && \text{(from Eq. (4.13))} \\
\mathbb{R} \ni \overline{v} &= \overline{e}\,\frac{\mathrm{d}e}{\mathrm{d}v} \\
&= -\,(y-e)\;\,h = \overline{e}\,h. && \text{(from Eq. (4.12))} \\
\mathbb{R} \ni \overline{h} &= \overline{e}\,\frac{\mathrm{d}e}{\mathrm{d}h} \\
&= \overline{e}\,(v). && \text{(from Eq. (4.12))} \\
\mathbb{R} \ni \overline{z} &= \overline{h}\,\frac{\mathrm{d}h}{\mathrm{d}z} \\
&= \overline{h}\,\sigma'(z). && \text{(from Eq. (4.11))} \\
\mathbb{R}^d \ni \overline{w} &= \overline{z}\,\frac{\mathrm{d}z}{\mathrm{d}w} \\
&= \overline{z}\,x. && \text{(from Eq. (4.10))} \\
\mathbb{R}^d \ni \overline{x} &= \overline{z}\,\frac{\mathrm{d}z}{\mathrm{d}x} \\
&= \overline{z}\,w. && \text{(from Eq. (4.10))}
\end{aligned}$$

Remark 4.1. An interesting mnemonic to remember backprop is to see that if the forward graph is

$$z = w_1 x_1 + w_2 x_2$$

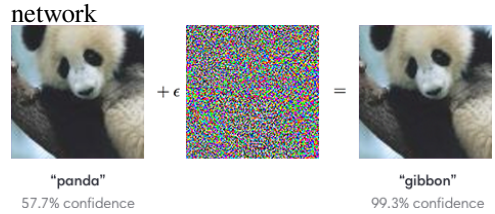
the backprop gradient is $\overline{w_1} = \overline{z}\,x_1$ and $\overline{w_2} = \overline{z}\,x_2$. If x_1 was large and dominated the computation of z during the forward propagation, then w_1 which is the multiplier of x_1 also gets a dominant share of the backprop gradient $\overline{z}$. The backprop gradient is shared equitably among the different quantities that took part in the forward computation. This is useful to remember when you build neural networks with complex architectures on your own: if there is a part of the network whose activations are very small and it is being combined with another part of the network whose activations have a large magnitude, then the former is not going to going to get a large enough backprop gradient.

Remark 4.2 (Gradient with respect to the input x). Notice that we obtain the gradient of the loss with respect to the input x

$$\frac{\mathrm{d}\ell}{\mathrm{d}x}$$

as a by-product of backpropagation. Backpropagation computes the gradient of the input activations to each layer $\overline{v}$ because this is precisely the gradient that is propagated downwards. So the gradient $\overline{x}$ should not be surprising, after all x is nothing but the input activation to the first layer. This gradient is useful, you can use to find what are called adversarial examples, i.e., input images which look like natural images to us humans but contain imperceptible noise that gives a large value of $\overline{x}$.

An example adversarial input to a deep network



4.2.2 Implementation of backpropagation

Consider our neural network classifier given by

$$f(x; v, S_1, \ldots, S_L) = \operatorname{sign}\left(w^\top \sigma\left(S_L^\top \ldots \sigma\left(S_2^\top\ \sigma(S_1^\top x)\right)\ldots\right)\right).$$

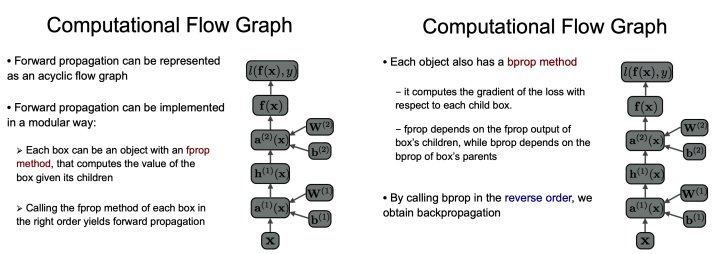


Figure 4.2: Schematic of forward and backward computations in backpropagation.

When you build such a multi-layer network in PyTorch, the kth layer is automatically equipped with two member functions.

```
def forward(self, h^{k-1}, S_k):
    # computes the output of the k^th layer
    # given output of previous layer h^k and
    # parameters of current layer S_k
    return h^k

def backward(self, h^k, d loss/dh^{k}, S_k):
    # computes two quantities
    # 1. d loss/d{S_k}
    # 2. d loss/d{h^{k-1}}
    return d loss/d{S_k}, d loss/d{h^{k-1}}
```

Such forward and backward functions exist for every layer, including the nonlinearities. If you implement a new type of layer in a neural network, say a new nonlinearity, you only need to write the forward function. The autograd module inside PyTorch automatically writes the backward function by looking at the forward function. This is why PyTorch is so powerful, you can build complex functions inside your deep networks without having to compute the derivatives yourself.

4.3 Weight initialization in fully-connected networks

We often talked about initializing the weights $w^{(0)}$ of a perceptron to zero when we looked at the dual perceptron,

$$w^* = \sum_{i=1}^{n} \alpha^i y^i x^i + w^{(0)}.$$

Setting $w^{(0)} = 0$ also produces an output of exactly zero in a deep network. We will see in Chapter 6 that this can be normalized appropriately using softmax to lead to a legitimate output (for a classifier zero weights after softmax would produce a equi-probable distribution on all the classes). But there are more important considerations in a non-convex optimization problem. For example:

1. **Scale symmetries** Suppose we have a network

$$f(x, w) = v^\top \sigma(S_2^\top \sigma(S_1^\top x)).$$

with a ReLU nonlinearity $\sigma(z) = \max(0, z)$. If we multiply all weights of the first layer by any scalar α and divide all weights of the second layer by the same scalar α, notice that the output does not change

$$\forall x, v, S_1, S_2 : \quad v^\top \sigma(S_2^\top \sigma(S_1^\top x)) = v^\top \sigma(\underbrace{\frac{S_2^\top}{\alpha}}_{S_2'^\top} \sigma(\underbrace{\alpha S_1}_{S_1'}{}^\top x)).$$

The reason for this is that the ReLU non-linearity is, what is called, positively homogeneous. This is a scale symmetry in the weights of a neural network. It entails that for any weight configuration w, there exists a direction (it is a straight line in our case) such that the output of the network does not change. Our weight initialization must be careful about these symmetries.

2. **Permutation symmetries** Next, suppose that $S_1 \in \mathbb{R}^{d \times p}$ and $S_2 \in \mathbb{R}^{p \times p}$. We can permute the columns of S_1 and correspondingly the rows of S_2 to keep the output unchanged. Formally, if T_π is a permutation matrix corresponding to a particular permutation π of the sequence $(1, 2, \ldots, p)$, then

$$\forall x, v, S_1, S_2 : \quad v^\top \sigma(S_2^\top \sigma(S_1^\top x)) = v^\top \sigma((\underbrace{T_\pi S_2}_{S_2'})^\top \sigma((\underbrace{S_1 T_\pi}_{S_1'})^\top x)),$$

again, for all data x (train or test) and weight configuration.

The choice $w^{(0)} = 0$ is bad in terms of both scale and permutation symmetries. It is the worst initialization for a deep network. For example, observe that if most weights are exactly zero, we need not even

⚠ Two important observations. First, the gradient of any surrogate objective on any data, train or test, along this symmetry is zero. Second, deep networks actually have fewer parameters than the total number of weights, due to these symmetries some weights are completely equivalent to each other (on all data, train or test).

permute all elements of the sequence $(1, 2, \ldots, p)$ to get a different weight configuration.

Exploding and vanishing gradients Observe that the backprop equations corresponding to Eq. (4.10)

$$z = w^\top x$$

are

$$\bar{w} = \bar{z}x$$
$$\bar{x} = \bar{z}w.$$

If $w \approx 0$, the derivative $\bar{x} = \frac{\partial \ell}{\partial x} \approx 0$. This can cause a numerical underflow. And if $\|w\| >> 1$, then we could also get a numerical overflow, especially because $\bar{z} = \bar{h}\sigma'(z) = v\bar{e}s'(z)$ which implies

$$\bar{x} = v\bar{e}s'(z)w.$$

In other words, the backprop derivative of the activations of lower layers depends upon the product of the weights on the layers above. If these weights take a large value (even at initialization, but in general at any time during training), then we can get numerical overflows.

> The weights of a deep network need to be initialized very carefully (not very small, not very large, and in general cognizant of the symmetries in the architecture). A poor choice of weight initialization can be detrimental to obtaining good generalization, even if it does not cause numerical issues. Ideas such as transfer learning, foundation models (self-supervised learning) are specific techniques to initialize the weights of a deep network.

4.3.1 Typical weight initialization schemes in deep learning

Consider a single neuron $z = w^\top x$ with $x \in \mathbb{R}^d$ and $w \in \mathbb{R}^d$. Suppose each dimension $x_i \sim N(0, \gamma^2)$. We are interested in initializing the weights w by drawing them independently from a Gaussian distribution

$$w_i \sim N(0, \sigma^2).$$

Our goal is to compute σ. Observe that

$$\begin{aligned}
\mathrm{E}[z] &= \sum_{i=1}^{d} \mathrm{E}\left[w_i x_i\right] \\
&= \sum_{i=1}^{d} \mathrm{E}[w_i]\,\mathrm{E}[x_i] \\
&= 0.
\end{aligned}$$

and

$$\begin{aligned}\mathrm{Var}[z] &= \mathrm{E}[z^2] - (\mathrm{E}[z])^2 \\ &= \sum_{i=1}^{d} \mathrm{E}\left[w_i^2 x_i^2\right] \\ &= \sum_{i=1}^{d} \mathrm{E}[w_i^2]\,\mathrm{E}[x_i^2] \\ &= d\sigma^2\gamma^2.\end{aligned}$$

In other words, while the mean of the pre-activation z is zero, its variance scales with the number of inputs to the neuron d. If we do not ensure that

$$d\sigma^2 = 1$$

then at higher layers, we will get even larger variances (because the variance of the input to the next layer now depends upon the number of inputs to the first layer, and the number of inputs to the second layer). Therefore, PyTorch sets the Gaussian to have

$$\sigma = \sqrt{\frac{1}{\#\text{ inputs to the layer}}}. \tag{4.14}$$

⚠ Actually, Pytorch initializes the weights using a uniform distribution supported between $[-1/\sqrt{d}, 1/\sqrt{d}]$. This distribution has a slightly smaller variance than the corresponding Gaussian. You can use torch.nn.init to initialize weighs in different ways.

Xavier initialization It is also important to think of a similar argument for the backward pass. Suppose we have a layer $z = S^\top x$. Since

$$\bar{x}_i = \sum_{j=1}^{p} S_{ji}\bar{z}_j,$$

we need to initialize each weight $S_{ij} \sim N(0, 1/\sqrt{p})$. While we cannot make the forward pass and the backward pass happy, we can strike a compromise by setting

$$\sigma = \sqrt{\frac{2}{\#\text{inputs } + \#\text{outputs}}}.$$

PyTorch actually draws weights from a uniform distribution supported on $\left[-\sqrt{\frac{6}{d+p}}, \sqrt{\frac{6}{d+p}}\right]$.

Bibliography

Balasubramanian, V. (2015). Heterogeneity and Efficiency in the Brain. *Proceedings of the IEEE*, 103(8):1346–1358.